Project Outline: CodeSentinel

A Self-Improving Multi-Agent Code Quality & Security Pipeline

---

1. Vision & Elevator Pitch

> *"CodeSentinel is a multi-agent AI system that autonomously writes, critiques, secures, tests, and explains code — iterating until every agent approves. It integrates directly into VSCode and runs on UiPath Maestro for enterprise-grade orchestration."*

The core insight: Every professional software team has developers, code reviewers, security auditors, and QA engineers. CodeSentinel gives every developer that entire team as AI agents, running in seconds.

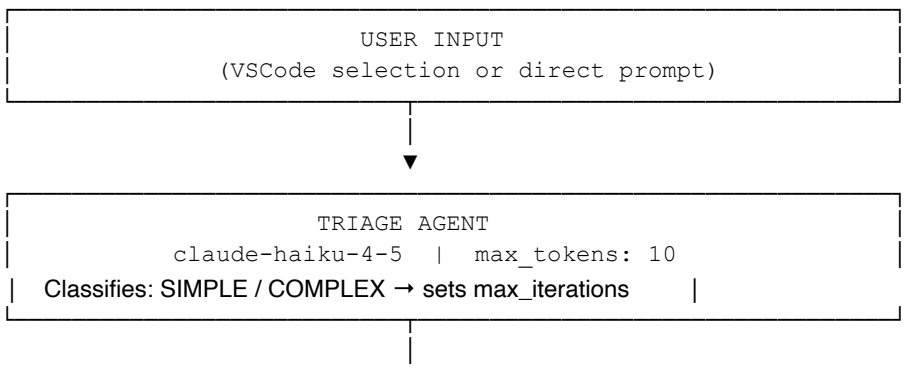
---

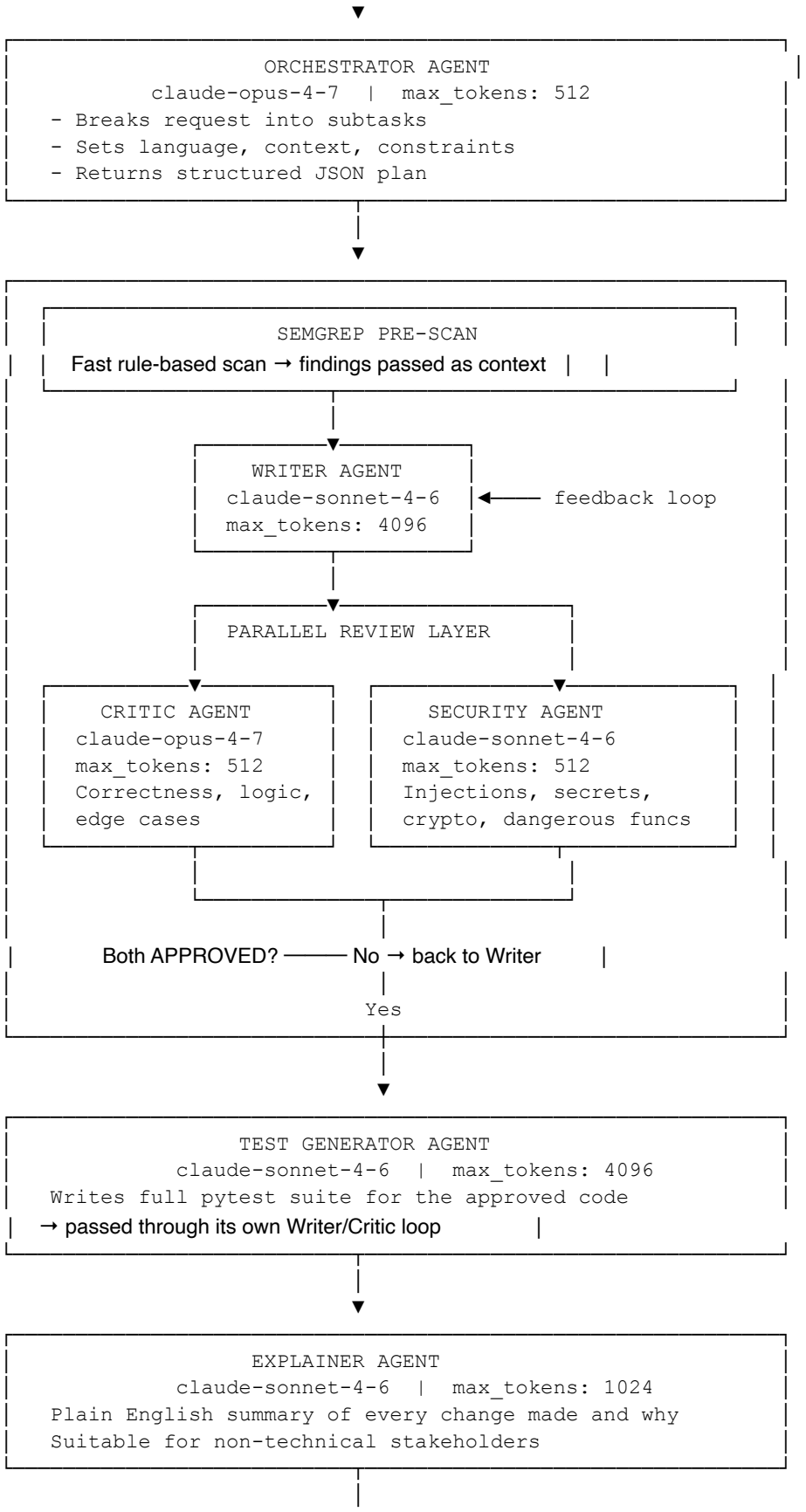
2. The Problem It Solves

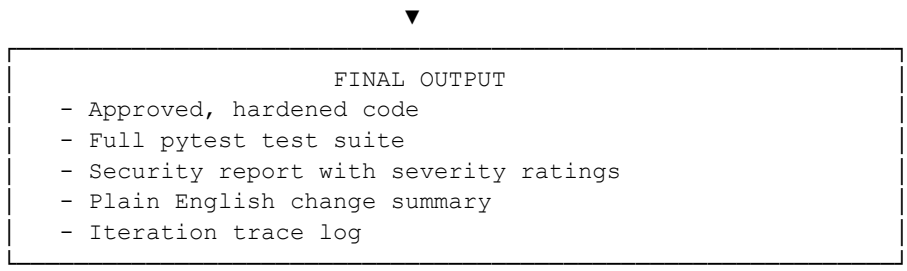
Problem	How CodeSentinel Solves It
Developers ship buggy code	Writer/Critic loop iterates until correctness is approved
Security vulnerabilities slip through	Dedicated security agent reviews every output
Writing tests is tedious	Test generator agent writes a full pytest suite automatically
Code reviews are slow and expensive	Entire review cycle runs in 10–15 seconds
Hard to see what AI is doing	Observability dashboard shows every agent decision in real time

---

3. Full Architecture




ORCHESTRATOR AGENT
claude-opus-4-7 | max_tokens: 512
- Breaks request into subtasks
- Sets language, context, constraints
- Returns structured JSON plan
SEMGREP PRE-SCAN
Fast rule-based scan → findings passed as context
WRITER AGENT
claude-sonnet-4-6
max_tokens: 4096
feedback loop
PARALLEL REVIEW LAYER
CRITIC AGENT
claude-opus-4-7
max_tokens: 512
Correctness, logic, edge cases
SECURITY AGENT
claude-sonnet-4-6
max_tokens: 512
Injections, secrets, crypto, dangerous funcs
Both APPROVED?
No → back to Writer
Yes
TEST GENERATOR AGENT
claude-sonnet-4-6 | max_tokens: 4096
Writes full pytest suite for the approved code
→ passed through its own Writer/Critic loop
EXPLAINER AGENT
claude-sonnet-4-6 | max_tokens: 1024
Plain English summary of every change made and why
Suitable for non-technical stakeholders



---

4. All Agents In Detail

Agent 1: Triage

- **Model:** `claude-haiku-4-5` (fastest, cheapest)
- **Input:** Raw code
- **Output:** `SIMPLE` or `COMPLEX`
- **Purpose:** Sets `max_iterations` to 2 or 5 — avoids wasting full loops on trivial code
- **Token limit:** 10

Agent 2: Orchestrator

- **Model:** `claude-opus-4-7`
- **Input:** User request + triage result
- **Output:** Structured JSON — language detected, constraints, subtasks
- **Purpose:** Frames the problem before the writer touches any code
- **Token limit:** 512

Agent 3: Writer

- **Model:** `claude-sonnet-4-6`
- **Input:** Request + Semgrep findings + previous code + combined critic/security feedback
- **Output:** Raw code only (no markdown)
- **Purpose:** Generates and iteratively fixes code based on all feedback
- **Token limit:** 4096
- **Optimizations:** Prompt caching on system prompt, streaming output

Agent 4: Critic

- **Model:** `claude-opus-4-7`
- **Input:** Current code
- **Output:** `APPROVED` or `ISSUES: - <issue>`
- **Purpose:** Adversarial correctness review — logic, edge cases, type handling
- **Token limit:** 512

- **Runs in parallel** with Security Agent

Agent 5: Security Agent

- **Model:** `claude-sonnet-4-6`
- **Input:** Current code + Semgrep pre-scan findings
- **Output:** `SECURE` or `VULNERABILITIES: - [CRITICAL/HIGH/MEDIUM] <issue>`
- **Purpose:** Dedicated security audit — injections, secrets, crypto, dangerous functions
- **Token limit:** 512
- **Runs in parallel** with Critic

Agent 6: Test Generator

- **Model:** `claude-sonnet-4-6`
- **Input:** Approved code
- **Output:** Full pytest suite
- **Purpose:** Writes tests grouped by category, runs through its own mini writer/critic loop
- **Token limit:** 4096

Agent 7: Explainer

- **Model:** `claude-sonnet-4-6`
- **Input:** Original code + final code + full iteration trace
- **Output:** Plain English summary of changes and why
- **Purpose:** Makes output readable for non-technical stakeholders and demo audiences
- **Token limit:** 1024

---

5. Performance Optimizations Built In

Optimization	Where Applied	Impact
Prompt caching	All agents	~90% cost reduction on repeated system prompts
Model tiering	Haiku → Sonnet → Opus by task complexity	~3x speed improvement
Parallel critic + security	Review layer	~2x faster per iteration
Triage early exit	Before pipeline starts	~2x faster on simple code
Streaming	Writer output	Feels instant in VSCode

Token caps per agent	Every agent	~20% faster, prevents verbose outputs
Semgrep pre-scan	Before writer loop	Fewer iterations needed

---

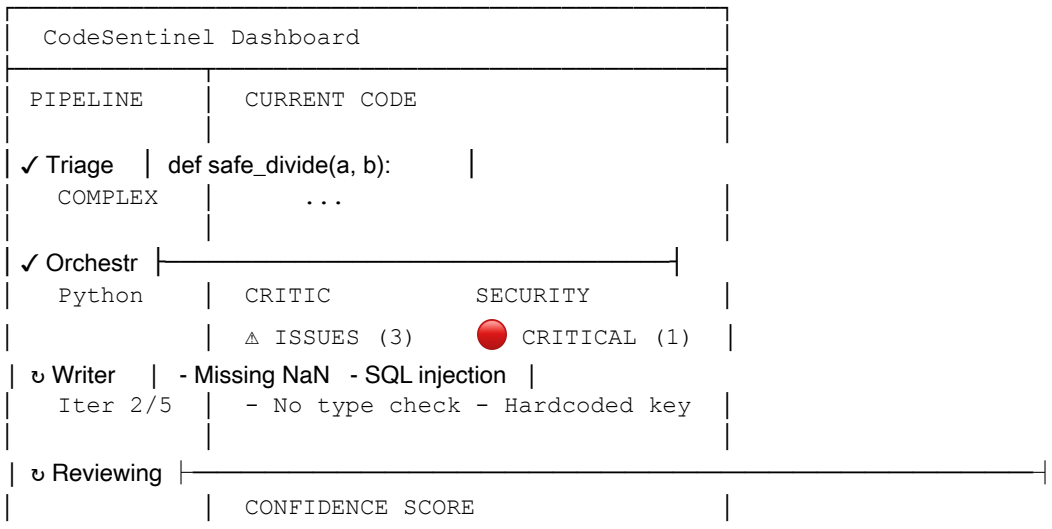
6. Tech Stack

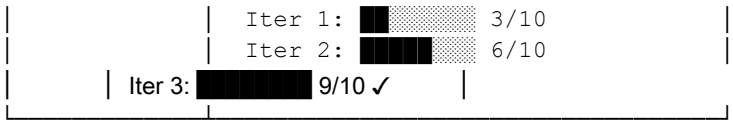
Layer	Technology
AI Models	Claude API (Haiku, Sonnet, Opus)
Security Pre-scan	Semgrep OSS
Parallelism	Python `concurrent.futures.ThreadPoolExecutor`
Environment	`python-dotenv`
Testing	`pytest`
VSCode Extension	TypeScript, Anthropic SDK
Dashboard	Streamlit
Hackathon Orchestration	UiPath Maestro
Observability	Structured JSON logs + Streamlit trace viewer

---

7. Streamlit Observability Dashboard

A real-time dashboard showing the pipeline running, designed specifically for the demo video:





---

8. VSCode Extension Flow

1. Developer selects any code
2. `Cmd+Shift+P` → **"CodeSentinel: Improve & Secure"**
3. Progress notification shows each agent working
4. Side-by-side diff view opens showing changes
5. Developer chooses: **Apply Changes** or **Discard**
6. Test file auto-created alongside the source file
7. Change summary shown in a VSCode panel

---

9. Demo Script (5 Minutes)

Time	What You Show
0:00–0:30	The problem — show obviously broken, insecure code
0:30–1:00	Trigger pipeline from VSCode — show progress notification
1:00–2:00	Streamlit dashboard — show critic finding bugs, security agent flagging CRITICAL vulnerability
2:00–3:00	Show iteration 2 — writer fixes everything, both agents approve
3:00–4:00	Show generated test suite — 40+ tests, all passing
4:00–4:30	Show explainer output — plain English summary
4:30–5:00	Pitch — "This is what every developer's team should look like"

---

10. Hackathon Positioning

Track: UiPath Test Cloud — *"Agents that reimagine software testing and quality assurance"*
Why you win:

- Only submission with a **self-correcting** multi-agent loop
- Only submission where **both correctness and security** must be approved
- Only submission that **generates and validates its own test suite**
- Only submission **integrated into VSCode** as a usable product
- Only submission with a **real-time observability dashboard**

---

11. 5-Week Build Timeline

Week	Goal
Week 1	Add security agent + Semgrep integration + parallel review layer
Week 2	Add triage agent + model tiering + prompt caching + token caps
Week 3	Build Streamlit dashboard with real-time trace view + confidence scores
Week 4	VSCode extension polish — diff view, explainer panel, streaming
Week 5	UiPath Maestro integration + demo video + GitHub docs + submission

---

12. File Structure (End State)

```
CodeSentinel/
├── agents/
│   ├── triage.py
│   ├── orchestrator.py
│   ├── writer.py
│   ├── critic.py
│   ├── security.py
│   ├── test_generator.py
│   └── explainer.py
├── pipeline.py          ← main orchestration
├── semgrep_scan.py      ← pre-scan integration
├── dashboard.py         ← Streamlit UI
├── vscode-extension/    ← TypeScript extension
├── tests/               ← meta-tests for the pipeline itself
├── demo/
```

```
│   ├── vulnerable_code.py  ← intentionally bad code for demo
│   └── demo_script.md
├── .env
├── requirements.txt
└── README.md
```